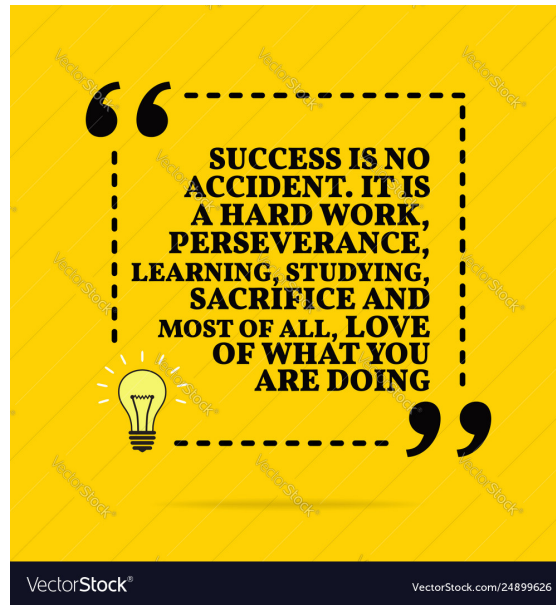




Lab Session on Hashing



Good
Morning

Content

01. Longest substring without repeat
02. First non repeating element
03. Check subarray with sum = 0
04. Check subarray with sum = k
05. Longest consecutive sequence

Problem statement → Time to implement code → Teach that particular question

Longest substring without repeat

Given a string str, find the longest substring without repeating character.

str = "a b c b d e a b" → Ans = 5

str = "b b b" → Ans = 1

str = "p w w k e w" → Ans = 3

Brute force →

01. Generate every possible substring of the string.
02. For a particular substring, check if it has all unique characters or not.

```
ans = 0
```

```
for (i = 0 → n) {
```

```
    for (j = i → n) {
```

```
        // substr = [i, j]
```

```
        HashSet <Character> hs
```

```
        for (k = i → j) {
```

```
            | hs.add(str.charAt(k));
```

```
            | if (hs.size() == j - i + 1) ans = Math.max(ans, j - i + 1);
```

Tc: $O(n^3)$

Sc: $O(n)$

Optimal Solution →

Steps:

- Initialize two pointers start and end to represent the current window.
- Use a hash set to keep track of characters in the current window.
- Move the end pointer to expand the window and include new characters.
- If a duplicate character is encountered, move the start pointer to reduce the window until the duplicate is removed.
- Update the maximum length of substrings without repeating characters as you expand the window.

str = a b c b d e a

↑
i

↑
j

e b c d
a

ans = 0 1 2 3 4 5

```
int i=0, j=0; ans = 0
```

```
HashSet<Character> hs = new HashSet<>();
```

```
while (j < str.length())
```

```
    char ch = str.charAt(j)
```

```
    while (i < j && hs.contains(ch))
```

```
        hs.remove(str.charAt(i));
```

```
        i++;
```

```
    hs.add(ch);
```

```
    if (j - i + 1 > ans)
```

```
        ans = j - i + 1;
```

```
    j++;
```

```
return ans;
```

Q what if I want maximum substring with repeating character.

```
int st = -1
```

```
int i = 0, j = 0; ans = 0
```

```
HashSet < Character > hs = new HashSet < > ();
```

```
while (j < str.length())
```

```
    char ch = str.charAt(j)
```

```
    while (i < j && hs.contains(ch))
```

```
        hs.remove(str.charAt(i));
```

```
        i++;
```

```
    hs.add(ch);
```

```
    if (j - i + 1 > ans)
```

```
        st = i;
```

```
        ans = j - i + 1;
```

```
    j++;
```

```
return str.substring(st, st + ans);
```

Problem 2 First non repeating element

Problem Statement

Given N elements, find the first non-repeating element.

Example

Input 1:

N = 6

1 2 3 1 2 5

Output1:

ans = 3

~~1 2 3 1 2 5~~

Input 2:

N = 8

4 3 3 2 5 6 4 5

Output 2:

ans = 2

Idea → Build hashmap for frequency & then
search for first non repeating character

```
HashMap<I, I> hm = new HashMap<>();
```

```
for (int ele : A) {
```

```
    | hm.put(ele, hm.getOrDefault(ele, 0) + 1);
```

```
for (int ele : A) {
```

```
    if (hm.get(ele) == 1) {
```

```
        return ele;
```

TC: $O(N)$

SC: $O(N)$

```
    },  
    return -1; // no unique element
```

* Subarray with sum = 0

Given an array containing N elements. Check if there exist a subarray with sum = 0.

$A[] = \{ \underset{0}{2} \ \underset{1}{2} \ \underset{2}{1} \ \underset{3}{-3} \ \underset{4}{4} \ \underset{5}{3} \ \underset{6}{1} \ \underset{7}{-2} \ \underset{8}{-3} \ \underset{9}{2} \} \rightarrow \underline{\underline{\text{true}}}$

Brute force $\rightarrow$ Generate all subarrays & then for a particular subarray, iterate & get sum of subarray.

TC: $O(N^3)$

SC: $O(1)$

Optimal Idea →

$A[] = \{ \underset{0}{2} \ \underset{1}{2} \ \underset{2}{1} \ \underset{3}{-3} \ \underset{4}{4} \ \underset{5}{3} \ \underset{6}{1} \ \underset{7}{-2} \ \underset{8}{-3} \ \underset{9}{2} \}$

$psum[] = \{ 2 \ 4 \ 5 \ 2 \ 6 \ 9 \ 10 \ 8 \ 5 \ 7 \}$



if ($psum[i]$ repeats)

the elements between last $psum$ & current $psum$ has contributed 0.

$A[] = \{ 2 \ 1 \ -3 \ 4 \}$

$psum[] = \{ 2 \ 3 \ 0 \ 4 \}$

↳ if $psum[i] == 0 \rightarrow$ one subarr with sum = 0

```
int sum = 0
```

```
HashSet<Integer> hs = new HashSet<>();
```

```
for (i=0; i<n; i++) {
```

```
    sum += A[i];
```

```
    if (sum == 0 || hs.contains(sum))
```

```
        return true;
```

```
    ,
```

```
    hs.add(sum);
```

TC : $O(N)$

SC : $O(N)$

OR

```
int sum = 0;
HashSet<Integer> hs = new HashSet<>();
hs.add(0);
for (i=0; i<n; i++) {
    sum += A[i];
    if (hs.contains(sum))
        return true;
    hs.add(sum);
}
```

$T_c: O(N)$
 $Sc: O(N)$

Problem 4 Subarray with Sum K

FIGQLUT

Problem Statement

Given an array $arr[n]$ check if there exists a subarray with $sum = K$

Example:

We have the following array

Index 0 1 2 3 4 5 6 7 8

arr[7] 2 3 9 -4 1 5 6 2 5

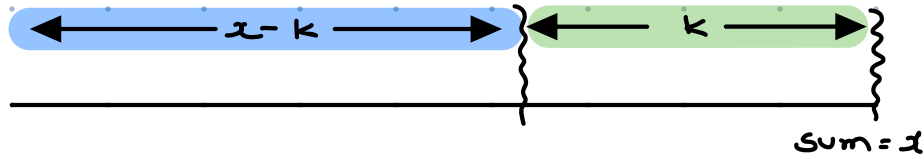
Possible subarrays for the following values of K are,

- $k = 11$: {2 3 9 -4 1}, {5, 6}
- $k = 10$: {2 3 9 -4}
- $k = 15$: {-4, 1, 5, 6, 2, 5}

Check if there exist a subarray with sum = 110 in the given array?

$A = [5, 10, 20, 100, 105]$

No



Idea $\rightarrow$ For a particular sum = x, we need to search if $(x-k)$ is present previously. If it is there, then this confirms we have a subarr with sum = x

$A[] = \{2, 3, 9, -4, 1, -6, 6, 2, 5\}$ $k = 6$

sum = 2 5 14 10 11 5 11
(x)

tar = -4 -1 8 4 5 -1 5
(x-k)

if (hs.contains(5))

$\Downarrow$

True

2	5	14
10	11	

```
int sum = 0;
```

```
HashSet<Integer> hs = new HashSet<>();
```

```
hs.add(0);
```

```
for (i=0; i<n; i++) {
```

```
    sum += A[i];
```

```
    if (hs.contains(sum-k))
```

```
        return true;
```

```
}
```

hs.add(sum);

TC: $O(N)$

SC: $O(N)$

* Longest Consecutive Sequence

Given an array of length N , find longest consecutive subsequence such that elements are consecutive integers

$A[] = \{1, 2, 100, 101, 3, 7, 9\}$

1 $\rightarrow$ 1 2 3

7 $\rightarrow$ 7

9 $\rightarrow$ 9

100 $\rightarrow$ 100 101

} Ans = 3

$A[] = \{7, 9, 11, 13, 8, 6, 4\}$

4 $\rightarrow$ 4

6 $\rightarrow$ 6 7 8 9

11 $\rightarrow$ 11

13 $\rightarrow$ 13

} Ans = 4

Approach

- The idea is to use Hashing.
- We first insert all elements in a Set.
- Then, traverse over all the elements and check if the current element can be a starting element of a consecutive subsequence.
- To check if the current element, say X can be a starting element, check if $(X - 1)$ is present in the set.
 - If $(X - 1)$ is present in the set, then X cannot be starting of a consecutive subsequence.
 - Else if $(X - 1)$ is not present, then start from X and keep on removing elements $X + 1, X + 2 \dots$ to find a consecutive subsequence.

```
HashSet<Integer> hs = new HashSet<>();
```

```
for (int ele : A) hs.add(ele);
```

```
for (int ele : A) {
```

```
    if (hs.contains(ele - 1)) continue;
```

```
    int end = ele;
```

```
    while (hs.contains(end))
```

```
        end = end + 1;
```

```
    ans = Math.max(ans, end - ele);
```

```
}
```

TC: $O(N)$
SC: $O(N)$